ChatGPT

System Design Prompt: Margros (Hyperlocal Restaurant Workforce & Vendor Marketplace)

Design **Margros** as a full-stack platform connecting **Restaurants**, **Workers**, and **Vendors** in a hyperlocal, admin-governed marketplace. Key flows and requirements include:

- **Role-Based Onboarding & Warm Funnel:** The homepage clearly segments users by role ("For Restaurants", "For Workers", "For Vendors"), allowing visitors to explore relevant sections (e.g. browse jobs or vendor categories) before committing to sign up [1]. Once a role is selected, present a tailored landing page. For example, restaurants see "Hire Staff" CTAs; workers see "Find Jobs"; vendors see service categories. Only after exploration is the signup required. The signup form collects email, password, and sends a one-time OTP for verification. (Users should be able to preview the platform in a read-only mode before signing up.)

- **Profile Completion (per Role):** After signup, guide users through a role-specific profile flow. For **Restaurants**, require fields like restaurant name, contact info, seating capacity, cuisine type, plus upload of a food license image. For **Workers**, collect their role (chef, waiter, etc.), years of experience, current location, availability schedule, and either an Aadhaar card image or proof of work experience. For **Vendors**, collect business proof (e.g. GST certificate), service category (e.g. kitchen equipment, cleaning supplies), and operating zone(s). (All profiles emphasize *"verified"* status once approved.)

- **AI-powered Document Verification:** Integrate Groq's multimodal Vision LLM to pre-screen uploaded documents via OCR [2]. For **restaurants**, the Vision LLM extracts fields from the food license image (owner name, license number, expiration date, address). For **workers**, it reads text from Aadhaar or employment certificates to verify identity and experience. For **vendors**, it extracts details from GST or registration images. Use these AI outputs to auto-flag inconsistencies (e.g. expired license) and pass the result to the admin. Importantly, this AI step is a *pre-screen* – all documents then go to a human admin for final approval. (Groq's API is specifically suited for fast OCR/image understanding [2].)

-

Admin Dashboard & Control Engine: Build a comprehensive admin panel as the heart of the platform. Admins should see an overview (total restaurants, active jobs, verified workers, pending approvals, etc.). Key features include:

- **Profile Approval:** A section listing pending restaurants, workers, and vendors. Admin can view each profile and **approve/reject** them. For workers, the admin can mark profiles as "verified," add notes, or block users [3]. For vendors, admins can approve or reject applications and track commission rates. (Detailed vendor lists show name, category, coverage zone, leads, conversion status.)
- **Zone & Hyperlocal Management:** A zone-management tool lets admin define geographic zones and assign restaurants/workers to them. Show zone stats (e.g. "Zone: Yelahanka – 40 restaurants, 300 workers, 12 open jobs") to balance supply-demand [4]. This "city-by-city" logic enforces the hyperlocal strategy: users see only matches within their zone.
- **Hiring & Placements Tracking:** Admin can view all job posts, active hires, and replacements. Track placement records (start date, role, guarantee period) to calculate metrics like churn rate and retention [3] [4].
- **Replacement & Dispute Management:** Handle restaurant replacement requests (auto-assign a new worker before deadline) and record exit reasons. Manage support tickets or disputes (worker complaints, payment issues), logging outcomes for quality.
- **Analytics & Growth:** Provide dashboards for key KPIs (avg time-to-hire, jobs per restaurant, revenue per zone, etc.) – essentially treating the admin panel as a "city-level workforce control room" [4].

- **Customer Engine & Vendor Network:** Control marketing campaigns (e.g. listing in apartment/college networks) and oversee the vendor marketplace. Vendors are displayed in curated categories with "quote request" features, but only after admin approval.

- **Two-Way Hiring Flows:** Support both sides of hiring. Restaurants can **post job requirements** (role, salary, shift, etc.) and review matched workers. Alternatively, restaurants can **browse worker profiles** (filter by role, experience, availability, zone). Workers can **browse open jobs** and apply or make themselves "actively available". This two-way engine ensures restaurants don't just wait for applicants – they can search vetted candidates. (The UI should easily let a worker view jobs or a restaurant search candidate lists.)

- **Unified User Journey:** The overall flow is: **Homepage → Select Role → Role Landing Page → Signup (email/password + OTP) → Complete Profile (upload docs) → Vision LLM Pre-Verify → Admin Review/Approval → Access Dashboard (role-specific).** Upon admin approval, restaurants see a hire-staff dashboard; workers see a jobs dashboard; vendors see leads and quotes.

- **Tech Stack & Architecture:** Use **Next.js** (React) hosted on Vercel for both frontend and backend (Edge/API routes) to enable serverless scalability. Store data in **Supabase** (PostgreSQL) using Supabase Auth for login/OTP and Supabase Storage for uploaded docs. This integration is turnkey: "Supabase on Vercel" provides a Postgres DB, Auth, Storage and realtime features with seamless deployment [5]. (One could also use Supabase Functions or Vercel Edge Functions for custom logic.) All APIs and pages should be implemented in Next.js, taking advantage of its performance and SEO.

- **Operational Design Principles:** The platform must emphasize **admin-first control** (every critical action is overseen by staff) and **verified quality**. For example, no worker is "active" until admin approves them [3]. Zones enforce **hyperlocal matching** (only nearby workers/vendors appear) [4]. Maintain a strict **verification pipeline**: vision-based OCR is used, but final authority is human. Ensure **two-way hiring** (both post-a-job and browse-candidates paths exist). Vendors are **curated and segmented** by category/zone and only shown after clearance.

Include these requirements in detail (with relevant UI flows and admin control needs) so the development team clearly understands Margros's end-to-end design. [3] [5] (Focus on robust admin controls, zone logic, user verification, and the specified tech stack.)

Sources: Relevant design patterns and admin-panel guidelines are documented in the Margros reference materials [3] [4] [5] [2].

---

[1] home page and restaurant owner.pdf
file://file_0000000023047207989b2bf85efd042d

[2] Images and Vision - GroqDocs
https://console.groq.com/docs/vision

[3] admin.pdf
file://file_00000000f3f87207bb5b6e73f68dfcac

[4] admin.pdf
file://file_00000000a30071fd8e4f1b383371cd84

[5] Supabase for Vercel
https://vercel.com/marketplace/supabase